

```

import os
import anthropic
from telegram import Update
from telegram.ext import Application, CommandHandler, MessageHandler, filters, Co

# Клиент Anthropic
client = anthropic.Anthropic(api_key=os.environ.get("ANTHROPIC_API_KEY"))

# Системный промпт — мозг агента
SYSTEM_PROMPT = """"Ты — AI агент для создания презентационного контента USDKG.

КОНТЕКСТ ПРОЕКТА:
- USDKG — золотообеспеченный стейблкоин, эмитент ОАО «ЭВА», Кыргызстан
- Лицензия Министерства финансов КР
- Текущая эмиссия: 50М токенов = $53.8М (386 кг золота)
- Цена золота: $4,331/oz
- Блокчейны: TRC-20 (47М) + ERC-20 (3М)
- Аудит: ConsenSys Diligence + Kreston Global
- Партнёры: CoinTR (Турция), Asterium (Узбекистан), RedStone DeFi
- OTC через лицензированный криптообменник (не CNE)
- DEX: Uniswap + Curve Finance

ПЛАН ЭМИССИИ:
- Конец 2026: $500М
- Конец 2027: $10В (~71.8 тонн золота)

GOLD DIVIDEND МОДЕЛЬ:
- Уровень 1: базовый резерв 1:1 (неприкосновенен)
- Уровень 2: буфер роста (прирост цены золота)
- Уровень 3: 50% буфера держателям раз в год, 50% остаётся
- Локап 6-12 месяцев
- При $10В эмиссии и росте золота 15%: $750М/год держателям

КОМИССИИ:
- Спред пула ликвидности: 0.1-0.2%
- Комиссия OTC-оператора: 0.05%
- Спред на импорте золота: 2% (чистый доход эмитента 1%)

ОТКРЫТЫЙ ИМПОРТ ЗОЛОТА В КР:
- Любой игрок ввозит золото → получает USDKG 1:1 минус 2%
- Стандарты LBMA
- Партнёрские хранилища: ОАЭ, Сингапур

ДИСТРИБЬЮЦИЯ:

```

- Сеть USDT-обменников как готовая инфраструктура
- Хавала-сеть как расчётный слой
- Guardian Service для VIP клиентов от \$1M
- Целевые рынки: СНГ, ОАЭ, Турция, Таиланд, Корея, Япония, Европа

СТИЛЬ ПРЕЗЕНТАЦИЙ (изучен из референсов):

- Тёмный или синий фон (#0A0C0F или #1A2FCA)
- Золотой акцент (#C9A84C)
- Цифры крупно – занимают 40% слайда
- Один слайд = одна идея + одна большая цифра
- Минимум текста, максимум смысла
- Буллеты короткие и ёмкие
- Таблицы сравнения для конкурентов
- Временные шкалы для роадмапа
- Потокоские схемы для процессов

ТВОЯ ЗАДАЧА:

Когда пользователь просит создать слайд или раздел презентации:

1. Определи тип слайда (титул, цифры, схема, таблица, буллеты)
2. Создай контент в стиле Gamma App – структурированный Markdown
3. Добавь в конце блок [GAMMA HINT] с подсказкой какой тип слайда выбрать в Gamma
4. Отвечай на русском, если не просят иначе

ФОРМАТ ОТВЕТА для слайдов:

[ЗАГОЛОВОК СЛАЙДА]

[ПОДЗАГОЛОВОК]

[КЛЮЧЕВАЯ ЦИФРА или ФАКТ]

- Буллет 1
- Буллет 2
- Буллет 3

[GAMMA HINT: выбери тип "Big number" / "Comparison" / "Timeline" / "Process flow"

Если просят полную презентацию – генерируй все слайды последовательно.

Если просят перевод – переводить точно сохраняя структуру.

Если просят объяснение – объясняй кратко и по делу. """

```
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        🌐 *USDKG Presentation Agent*\n\n"
        "Я создаю контент для презентаций USDKG в твоём стиле.\n\n"
```

```

        "*Что я умею:*\n"
        "• Создать слайд по теме\n"
        "• Написать полный раздел презентации\n"
        "• Перевести контент на английский/китайский\n"
        "• Объяснить любой аспект USDKG\n\n"
        "*Примеры запросов:*\n"
        "— Сделай слайд про Gold Dividend\n"
        "— Напиши раздел про архитектуру дистрибьюции\n"
        "— Создай слайд сравнения USDKG vs USDT\n"
        "— Переведи на английский: [текст]\n\n"
        "Пиши что нужно 📌",
        parse_mode="Markdown"
    )

```

```

async def help_command(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "*Команды:*\n"
        "/start — начало работы\n"
        "/help — эта справка\n"
        "/full — полная презентация USDKG\n"
        "/investor — питч для инвестора\n"
        "/exchanger — питч для обменников\n\n"
        "*Или просто напиши запрос на русском*",
        parse_mode="Markdown"
    )

```

```

async def full_presentation(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("⌚ Генерирую полную презентацию... (~1 минут

    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=4000,
        system=SYSTEM_PROMPT,
        messages=[{
            "role": "user",
            "content": "Создай полную структуру презентации USDKG для инвестора.
        }]
    )

    text = response.content[0].text
    # Разбиваем на части если длинный
    if len(text) > 4000:
        parts = [text[i:i+4000] for i in range(0, len(text), 4000)]
        for part in parts:
            await update.message.reply_text(part, parse_mode="Markdown")

```

```

else:
    await update.message.reply_text(text, parse_mode="Markdown")

async def investor_pitch(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("⌚ Генерирую питч для инвестора...")

    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=3000,
        system=SYSTEM_PROMPT,
        messages=[{
            "role": "user",
            "content": "Создай короткий питч для инвестора — 8 слайдов. Фокус на
        }]
    )

    await update.message.reply_text(response.content[0].text, parse_mode="Markdow

async def exchanger_pitch(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("⌚ Генерирую питч для обменников...")

    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=2000,
        system=SYSTEM_PROMPT,
        messages=[{
            "role": "user",
            "content": "Создай питч для обменников — почему им выгодно работать с
        }]
    )

    await update.message.reply_text(response.content[0].text, parse_mode="Markdow

async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_text = update.message.text

    await update.message.reply_text("⌚ Генерирую...")

    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=2000,
        system=SYSTEM_PROMPT,
        messages=[{
            "role": "user",

```

```

        "content": user_text
    }]
)

text = response.content[0].text

if len(text) > 4000:
    parts = [text[i:i+4000] for i in range(0, len(text), 4000)]
    for part in parts:
        await update.message.reply_text(part, parse_mode="Markdown")
else:
    await update.message.reply_text(text, parse_mode="Markdown")

def main():
    token = os.environ.get("TELEGRAM_BOT_TOKEN")
    if not token:
        raise ValueError("TELEGRAM_BOT_TOKEN не задан")

    app = Application.builder().token(token).build()

    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("help", help_command))
    app.add_handler(CommandHandler("full", full_presentation))
    app.add_handler(CommandHandler("investor", investor_pitch))
    app.add_handler(CommandHandler("exchanger", exchanger_pitch))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_message))

    print("Бот запущен...")
    app.run_polling()

if __name__ == "__main__":
    main()

```